

Itc assignment

Name: Muskan Khanwani
ERP:35610

Personalization block:

$R = 10$

$V = 2$

$B = 1010$

$N = 3$

GitHub repository:

Personalization Parameters

ERP Number: 35610

R Value: 10

Variant Number (V):

$$10 \bmod 4 = 2$$

Target Modulus $M = 5$ (Master Variant Table for $V = 2$)

Personalized Input String (B): {"1010"}

Question 1:

Task A:

the turing machine recognises binary strings that are divisible by M by reading string from left to right.

the machine is divided into 5 states q_0, q_1, q_2, q_3, q_4 .

when the tape reads a binary bit either zero or one, it multiplies that bit with the previous remainder and adds the bit.

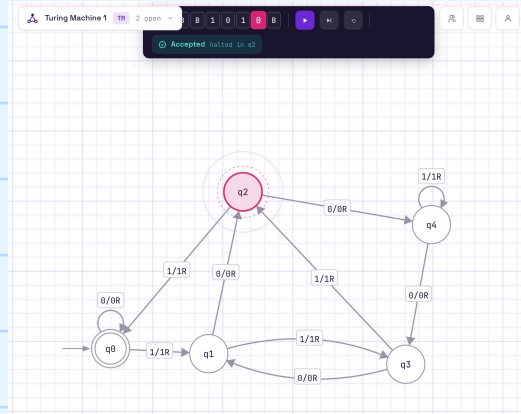
This number is divided and the remainder determines which state we should Move to.

The remainder keeps getting updated and the process of calculation continues until remainder becomes zero.

When remainder becomes zero we move to first state and accept the string.

Task B:

Current state	Read bit	Calculation	Division	Remainder	Next state
q_0	0	$(2 \times 0) + 0 = 0$	$0 \div 5 = 0$	0	q_0
q_0	1	$(2 \times 0) + 1 = 1$	$1 \div 5 = 0$	1	q_1
q_1	0	$(2 \times 1) + 0 = 2$	$2 \div 5 = 0$	2	q_2
q_1	1	$(2 \times 1) + 1 = 3$	$3 \div 5 = 0$	3	q_3
q_2	0	$(2 \times 2) + 0 = 4$	$4 \div 5 = 0$	4	q_4
q_2	1	$(2 \times 2) + 1 = 5$	$5 \div 5 = 1$	0	q_0
q_3	0	$(2 \times 3) + 0 = 6$	$6 \div 5 = 1$	1	q_1
q_3	1	$(2 \times 3) + 1 = 7$	$7 \div 5 = 1$	2	q_2
q_4	0	$(2 \times 4) + 0 = 8$	$8 \div 5 = 1$	3	q_1



Task c:

```

{"nodeMap":{"q0":{"id":"q0","x":373.72864445975756,"y":1111.364447059846},"q1":
{"id":"q1","x":527.8849244675562,"y":1119.3102879597063},"q2":
{"id":"q2","x":581.6383347861627,"y":865.4518278857013},"q3":
{"id":"q3","x":837.3909987355222,"y":1135.9585756895037},"q4":
{"id":"q4","x":877.7598366984689,"y":916.8265770061525}},"transitionMap":{"q0":
[{"label":"1/1R","sourceid":"q0","targetid":"q1"},{"label":"0/0R","sourceid":"q0","targetid":"q0"}],"q1":
[{"label":"0/0R","sourceid":"q1","targetid":"q2"},{"label":"1/1R","sourceid":"q1","targetid":"q3"}],"q2":
[{"label":"1/1R","sourceid":"q2","targetid":"q0"},{"label":"0/0R","sourceid":"q2","targetid":"q4"}],"q3":
[{"label":"0/0R","sourceid":"q3","targetid":"q1"},{"label":"1/1R","sourceid":"q3","targetid":"q2"}],"q4":
[{"label":"0/0R","sourceid":"q4","targetid":"q3"},
{"label":"1/1R","sourceid":"q4","targetid":"q4"}]},"finalNodes":
["q0"],"nodeNum":5,"automataType":"TM","inputString":".100","stageProps":
{"x":-121.55414762154601,"y":-580.1000280253215,"scale":1,"draggable":true},"stateOutput":
{},"tapeMode":"single","inputString2":""}
  
```

Task D:

A Turing machine is designed or hardcoded for $m=5$ so it will have exactly 5 remainder states that will track the remainder.

This machine will not work for $m=3$ because it would only need 3 states

Question 2:

Task A;

The Turing Machine will process the input tape from left to right to recognize the specific target sequence consisting of six 'a' symbols followed by three 'b' symbols (aaaaaabbbb).

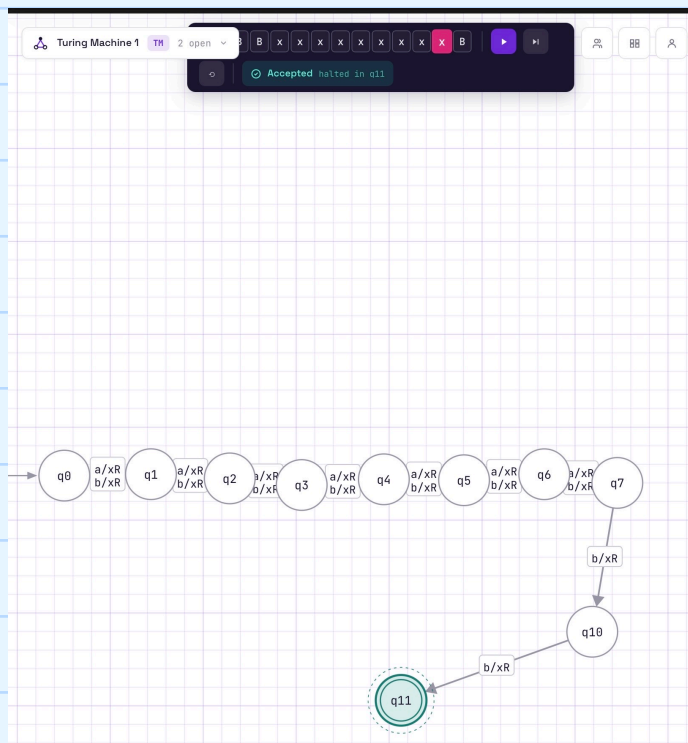
The machine starts at state q_0 at the leftmost tape head position. It one by one reads six consecutive 'a' symbols, when reading the sixth 'a', the machine expects three consecutive 'b' symbols, Finally, the machine reads the blank symbol to indicate the end of the string and moves into accept state. If any invalid symbol or out-of-order sequence is found at any stage, the machine halts and moves to reject state

Task B

State	Read	Write	Move Direction	Next state	Description
q_0	a	a	R	q_1	1st a
q_1	a	a	R	q_2	2nd a
q_2	a	a	R	q_3	3rd a

q ₂	a	a	R	q ₃	4th	a
q ₃	a	a	R	q ₄	5th	a
q ₄	a	a	R	q ₅	6th	a
q ₅	a	a	R	q ₆	1st	b
q ₆	b	b	R	q ₇	2nd	b
q ₇	b	b	R	q ₈	3rd	b
q ₈	b	b	R	q ₉	String accepted	
q ₉	#	#	R	Accept		

Task c:



Task D:

This machine prevents counting and off-by-one errors through a strict method where every state demands an exact expected symbol. If a string contains only five 'a's (eg = aaaaabbb), the machine reaches state q₅ expecting the sixth 'a', but finds 'b' instead, and moves to reject state. Similarly, if an extra 'b' is attached to the end (e.g., aaaaaabbbbb), state q₉ finds 'b' instead of the blank symbol and hence fails the final check and enters reject state. This ensures that only the exact sequence of six 'a's and three 'b's is accepted.

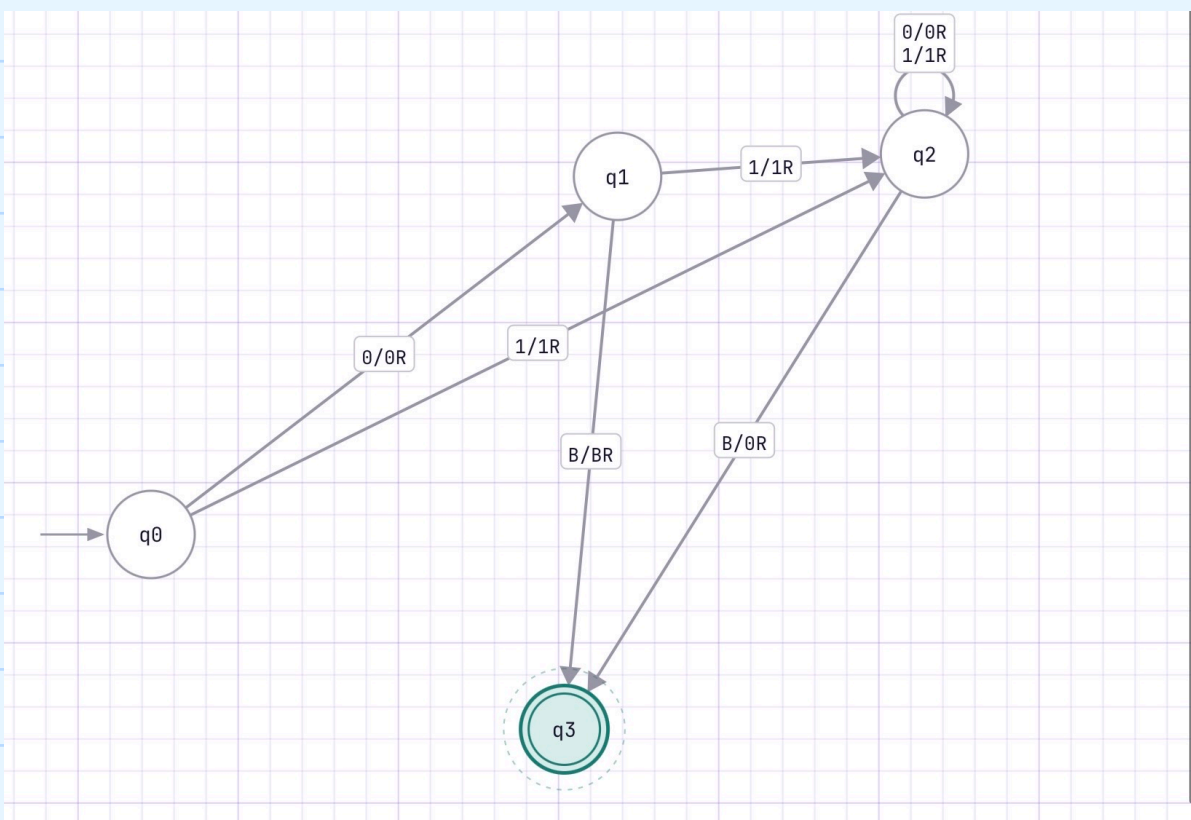
Question 3:

Task A:

Multiplying a number by 2 is equivalent to adding 0 at the last of each binary string .

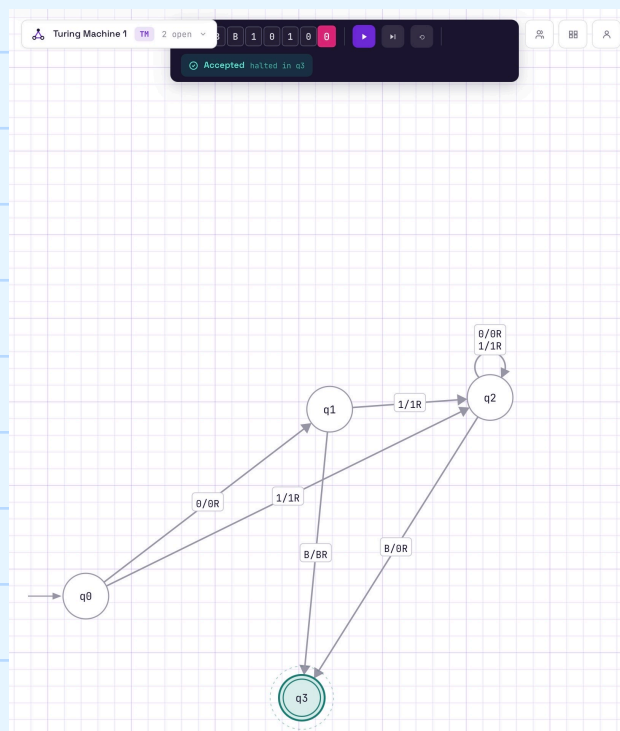
The machine reads each binary digit starting from the left and when reads a blank symbol writes 0 at that position and enters accept state

Task b:



Task C:

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 160.142377814903,
      "y": 2565.5909584588635
    },
    "q1": {
      "id": "q1",
      "x": 480.5387992090511,
      "y": 2319.6421336355957
    },
    "q2": {
      "id": "q2",
      "x": 691.3480843015765,
      "y": 2304.256077824584
    },
    "q3": {
      "id": "q3",
      "x": 443.96770110337,
      "y": 2699.324134153061
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "0/0R",
        "sourceid": "q0",
        "targetid": "q1"
      },
      {
        "label": "1/1R",
        "sourceid": "q0",
        "targetid": "q2"
      }
    ],
    "q1": [
      {
        "label": "B/BR",
        "sourceid": "q1",
        "targetid": "q3"
      },
      {
        "label": "1/1R",
        "sourceid": "q1",
        "targetid": "q2"
      }
    ],
    "q2": [
      {
        "label": "0/0R, 1/1R",
        "sourceid": "q2",
        "targetid": "q2"
      },
      {
        "label": "B/0R",
        "sourceid": "q2",
        "targetid": "q3"
      }
    ],
    "finalNodes": ["q3"],
    "nodeNum": 4,
    "automataType": "TM",
    "inputString": "1010",
    "stageProps": {
      "x": 80.33998945901908,
      "y": -2728.600415957813,
      "scale": 1,
      "draggable": true
    },
    "stateOutput": {
      "tapeMode": "single",
      "inputString2": ""
    }
  }
}
```



Task D:

Edge case is 0 because even when 0 is entered it moves to q2 even then one more zero is added the end and hence multiplication takes place

Question 4:

Task A:

The machine can recognize palindromes by comparing characters from the outer positions and then inner positions. First, the tape head reads the leftmost symbol and marks it with a unique symbol while moving to a state that remembers whether an a or b was read.

Next, the tape head moves all the way to the right end of the string until it finds a blank symbol, indicating the string has ended and then steps one position back to read the rightmost symbol.

this symbol matches the character remembered by the current state, the tape head marks it as blank unique symbol as well. If there is a mismatch, the machine immediately halts and rejects. If it matches, the tape head rewinds back to the next leftmost unread symbol and repeats the process. This loop continues until either all symbols are read and erased for even-length strings)

Task b:

State	Read	Write	Move	Next state
q_0	a	#	R	q_a
q_0	b	#	R	q_b
q_0	#	#	R	q_{accept}
q_a	a/b	a/b	R	q_a
q_a	#	#	L	$q_{match a}$
q_b	a/b	a/b	R	q_b
q_b	#	#	L	$q_{match b}$
$q_{match a}$	a	#	L	q_{rewind}
$q_{match a}$	b	b	-	q_{reject}
$q_{match a}$	#	#	R	q_{accept}
$q_{match b}$	a	a	-	q_{reject}
$q_{match b}$	b	#	L	q_{rewind}
$q_{match b}$	#	#	R	q_{accept}

q_{rewind} a/b a/b L q_{rewind}
 q_{rewind} $\#$ $\#$ R q_0

Task D:

A Turing machine needs multiple left-right passes instead of a single pass because it can only remember one symbol at a time in its state memory.

In a single left-to-right pass, once the tape head reaches the right end to match the first character, it cannot move backward to check the rest of the string. To verify a palindrome, the machine must erase one pair at a time, move all the way to the right to match it, and then come back to the left boundary to pick the next symbol.

Without these repeated left-and-right passes, the machine cannot check the middle characters of the string.

Question 5:

Task A:

In Question 5, the machine compares symbols that are separated by the delimiter #, meaning there are symbols on both the left and right sides of the #.

The tape head reads the leftmost unmarked symbol on the left side of #, marks it with a distinct symbol (like X), and enters a specific state (for example, q_foundb or q_foundb to remember what character was read).

The tape head then moves right, skipping any remaining unmarked symbols on the left side, and passes over the # delimiter.

After crossing #, if there are any previously marked symbols on the right side, it skips them as well until it finds the first matching unmarked symbol.

Once it finds the corresponding matching symbol, it marks it with a matching symbol (like Y) and rewinds all the way back to the left until it hits the last marked symbol X.

The machine repeats this loop until all symbols on the left are marked. Finally, it sweeps right past # to verify that no unmatched extra symbols remain on the right side before moving to q_accept state

Task b:

State	Read	Write	Move	Next state
q ₀	a	X	R	q ₁
q ₀	#	#	reject	q _{-rej}
q ₁	a	a	R	q ₁
q ₁	#	#	R	q _{searchb}
q _{searchb}	Y	Y	R	q _{searchb}
q _{searchb}	b	Y	L	q ₂
q _{searchb}	#	#	L	q ₃
q ₂	a/#/Y	a/#/Y	L	q ₂
q ₂	X	X	R	q ₀

q ₃	#/γ	#/γ	L	q ₃
q ₃	X	X	R	q ₄
q ₄	a	a	R	q _{accept}
q ₄	#	#	-	q _{rej}

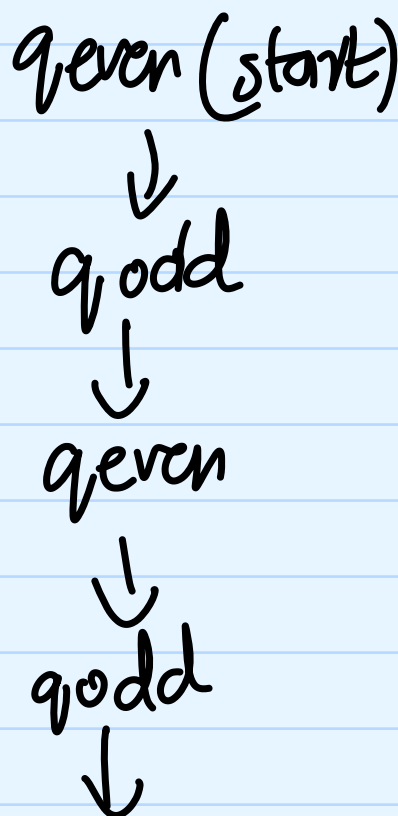
Task D:

a single left-to-right pass is not enough because the Turing machine cannot count of a's in memory. To compare a's and b's without a counter, the machine must match them one-by-one across the '#' symbol. It marks one 'a' on the left, moves to the right to mark its matching 'b', and then comes back to the left boundary to find the next unmarked 'a'. These continuous left-right swipes are necessary so the machine can return to the left side and verify if any extra 'a's remain after all 'b' are erased.

Question 6

Task A:

for tracing empty string and "11"



even two empty string and 11 has even no of 1s still rejected.

qreject

Task b:

→ Row 3 should read blank and accept the string
→ row 6 should read blank and accept the string.